

Gradient Descent

Gradient descent is an optimization algorithm used to find the minimum of a function. In machine learning, it's used to find the values for the model's **parameters** (e.g., w and b) that result in the lowest possible cost. It's a foundational algorithm used to train everything from simple linear regression models to advanced neural networks.

The Goal: Minimizing the Cost Function

The objective of gradient descent is to systematically find the parameters w and b that **minimize** the cost function $J(w, b)$.

The algorithm works for any cost function, including more complex ones with many parameters $(w_1, w_2, \dots, w_n, b)$.

How it Works: The Downhill Analogy

Imagine the cost function J as a hilly landscape, where the height of the ground represents the cost. The goal is to find the bottom of a valley.

1. **Start Somewhere:** Begin by making an initial guess for the parameters w and b . A common starting point is to set them all to zero. This places you at a specific point on the hilly landscape.
2. **Look Around:** From your current position, look around 360 degrees and determine which direction is the **steepest path downhill**.
3. **Take a Step:** Take a small step in that direction of **steepest descent**. You are now at a new, slightly lower point on the hill.
4. **Repeat:** Continue repeating this process—look for the steepest path downhill and take another small step.

By repeatedly taking small steps in the steepest downhill direction, you will eventually converge to the bottom of a valley, which represents a minimum of the cost function.

An Important Property: Local Minima

For complex, non-bowl-shaped cost functions (like those for neural networks), there can be multiple valleys, known as **local minima**.

- The specific local minimum that the algorithm finds **depends on the starting point**.
- Starting at a slightly different initial position can cause the algorithm to descend into a completely different valley.

- *Note:* The squared error cost function for linear regression is a simple **bowl shape** (convex), so it only has **one global minimum**. This means that no matter where you start, gradient descent will always lead to the same optimal solution.

The Gradient Descent Algorithm

Gradient descent is an iterative process that repeatedly updates the model parameters w and b to minimize the cost function $J(w, b)$.

The core of the algorithm is the following update rule, which is repeated until the model converges:

$$w := w - \alpha \frac{\partial}{\partial w} J(w, b)$$
$$b := b - \alpha \frac{\partial}{\partial b} J(w, b)$$

Components of the Update Rule

Let's break down what each part of the rule means:

- **$:=$ (The Assignment Operator):** In this context, the equal sign means we are **assigning** a new value to the parameter. We calculate the entire expression on the right and use it to update the value of w or b on the left.
 - **α (Alpha, The Learning Rate):**
 - Alpha is a small positive number (e.g., 0.01) that controls **how big of a step** you take downhill on each iteration.
 - A large α means taking aggressive, large steps.
 - A small α means taking small, cautious baby steps.
 - Choosing a good value for α is a critical part of making the algorithm work well.
 - **$\frac{\partial}{\partial w} J(w, b)$ (The Derivative Term):**
 - You don't need to know calculus to understand its role here.
 - This term calculates the **direction of the steepest descent**. It tells the algorithm *which way* to go to reduce the cost most effectively.
 - Multiplying the direction (derivative) by the step size (α) determines the overall update for the parameter.
-

Correct Implementation: Simultaneous Updates

A crucial detail for implementing gradient descent correctly is to perform **simultaneous updates** for all parameters. This means you must calculate the new values for w and b *before* updating either of them.

- **Correct Method:**
 1. Calculate the update for w and store it in a temporary variable: `temp_w = w - ...`

2. Calculate the update for b and store it in another temporary variable: $\text{temp_b} = b -$
...
3. Update w and b with the values from the temporary variables: $w = \text{temp_w}, b = \text{temp_b}$.

- **Incorrect Method (Sequential Update) ❌:**

1. Calculate the new w and immediately update it.
2. Use this **newly updated** w to calculate the update for b .
 - This is technically a different algorithm and is not the standard implementation of gradient descent.

Gradient Descent Intuition

To understand how gradient descent works, we can analyze the two key components of its update rule: the **derivative** and the **learning rate**.

The Role of the Derivative ($\partial J / \partial w$)

Let's use a simplified cost function $J(w)$ with only one parameter, w , to visualize the process.

- The **derivative** at any point on the cost function curve is the **slope of the tangent line** at that point.
- The slope tells us which direction is "downhill" and helps the algorithm decide whether to increase or decrease the parameter w .

There are two main scenarios:

Case 1: The slope is positive

1. This happens when our current value of w is to the right of the minimum. The tangent line points up and to the right.
2. The update rule is: $w := w - \alpha \cdot (\text{positive number})$.
3. Since α is also positive, we are subtracting a positive value from w .
4. This causes w to **decrease**, moving it to the left and closer to the minimum.

Case 2: The slope is negative

1. This happens when our current value of w is to the left of the minimum. The tangent line points down and to the right.
2. The update rule is: $w := w - \alpha \cdot (\text{negative number})$.
3. Subtracting a negative number is the same as adding a positive number.
4. This causes w to **increase**, moving it to the right and closer to the minimum.

In both cases, the derivative term automatically guides the parameter update in the correct direction to lower the cost.

The Learning Rate (α)

The **learning rate**, denoted by the Greek letter α (alpha), is a critical parameter in the gradient descent algorithm. It controls the size of the steps taken on each iteration, and choosing an appropriate value is essential for the algorithm's performance.

If the Learning Rate is Too Small 🐢

- **Behavior:** The algorithm takes extremely small "baby steps" on each iteration.
 - **Consequence:** Gradient descent will work, but it will be very **slow to converge**. It may take an excessive number of iterations to reach the minimum.
-

If the Learning Rate is Too Large 🚀

- **Behavior:** The algorithm takes huge steps, which can cause it to "overshoot" the minimum of the cost function. With each step, it might land on a point with an even higher cost than the previous one.
 - **Consequence:** Gradient descent may **fail to converge** or even **diverge**, meaning the cost gets worse over time, moving further and further away from the minimum.
-

Behavior at a Local Minimum ✅

- When the algorithm reaches a local minimum, the slope of the tangent line to the cost function is **zero**.
 - This means the **derivative term in the update rule is zero**.
 - The update becomes: $w := w - \alpha \cdot 0$, which simplifies to $w := w$.
 - **Consequence:** The parameter w stops changing. The algorithm has successfully **converged** and will remain at the local minimum.
-

Natural Decrease in Step Size

- Even with a **fixed learning rate α** , the step sizes taken by gradient descent will **automatically get smaller** as the algorithm approaches a minimum.
- **Why?** As you get closer to the bottom of the "bowl," the slope of the cost function becomes less steep. This means the derivative term gets smaller, resulting in a smaller update to the parameters.
- This property allows the algorithm to take large steps when it's far from the minimum and smaller, more precise steps as it gets closer, helping it converge smoothly.

Gradient Descent for Linear Regression

By combining the gradient descent algorithm with the specific squared error cost function for linear regression, we get a complete algorithm for training the model.

The Complete Algorithm

The final algorithm involves repeatedly performing the following **simultaneous updates** for w and b until the model converges.

- **Update for w :**

$$w := w - \alpha \left[\frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}) x^{(i)} \right]$$

- **Update for b :**

$$b := b - \alpha \left[\frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}) \right]$$

Key Components:

- $f_{w,b}(x^{(i)})$ is the model's prediction, calculated as $w x^{(i)} + b$.
 - $(f_{w,b}(x^{(i)}) - y^{(i)})$ is the error for the i -th training example.
-

The Derivative Terms (Calculus Derivation Optional)

The specific formulas in the update rules come from calculating the partial derivatives of the squared error cost function, $J(w, b)$.

- **Derivative with respect to w :**

$$\frac{\partial}{\partial w} J(w, b) = \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}) x^{(i)}$$

- **Derivative with respect to b :**

$$\frac{\partial}{\partial b} J(w, b) = \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})$$

You don't need to know calculus to implement the algorithm; you only need to use these final formulas.

A Special Property: Convexity

The squared error cost function used for linear regression has a special and very useful property: it is a **convex function**.

- **What this means:** The cost function has a single **bowl shape** and therefore has **no local minima**, only **one global minimum**.
- **Why it's important:** Because the cost function is convex, gradient descent is **guaranteed**

to converge to the global minimum (the best possible solution), as long as the learning rate α is chosen appropriately. It will never get "stuck" in a suboptimal valley.

Running Gradient Descent: The Algorithm in Action

This section visualizes the step-by-step process of gradient descent finding the optimal parameters for a linear regression model.

The Iterative Process Visualized

1. **Initialization:** We start with an initial guess for the parameters w and b . This corresponds to a specific starting point on the cost function's contour plot and a poorly fitting line on the data plot.
2. **Taking Steps:**
 - With each iteration (or step) of gradient descent, the algorithm calculates the direction of steepest descent and updates w and b .
 - On the contour plot, you can see the point representing (w, b) move closer to the center (the minimum).
 - Simultaneously, on the data plot, the straight line $f(x)$ adjusts its slope and intercept, providing a slightly better fit to the data with each step.
3. **Convergence:**
 - This process is repeated, with the cost $J(w, b)$ decreasing at each step.
 - Eventually, the algorithm **converges** at the **global minimum**—the center of the contour plot.
 - The final w and b values at this minimum define the model that best fits the training data. This final model can then be used to make predictions on new data.

Batch Gradient Descent

- The specific version of the algorithm we have been using is called **Batch Gradient Descent**.
 - **Definition:** The term "batch" means that on **every single step** of gradient descent, we look at the **entire training set** (all m examples) to calculate the derivatives. This is reflected in the summation term, $\sum_{i=1}^m$, which runs over all the examples.
 - *Note:* Other versions of gradient descent exist that look at smaller subsets of data, but for now, we are using the "batch" approach.
-